Verified Binary Decoding with a Domain-Specific Language: An Empirical Study of EFCH Lang against C, Rust–Kani, and Dafny

Erik Fisher Cueva Huari
Independent Researcher, VentureSynaptic Labs, Peru
ORCID: 0000-0001-7268-5510

Research manuscript, version 0.2 — 2 November 2026
Not peer reviewed; evidence reported from EXP-004, EXP-005, EXP-007–EXP-010 and EXP-015

Abstract

Binary-format parsers combine compact implementation requirements with subtle boundary, overflow, and canonicality constraints. This paper describes and evaluates EFCH Lang, a narrow domain-specific language that translates bounded binary-processing programs into C and checks their equivalence to explicit functional specifications. We report two archived experiments rather than claiming general language superiority. In an eight-function benchmark (EXP-005), all eight functions were represented and verified under the recorded protocol; generated C achieved a geometric mean runtime ratio of 0.90 against handwritten C on the measured workloads. Comparison against Rust with Kani and Dafny initially satisfied a preregistered viability rule, but subsequent adversarial analysis showed that effort comparisons were representation-sensitive, and that a simple Dafny specification-only implementation erased the purported effort advantage. A separate two-function experiment (EXP-015) found runtime ratios of 1.027 for IPv4 options and 0.966 for unsigned varint against independently authored C; both passed their specified tests and bounded verification checks. We also report the negative and mixed experiments of the same research line: a DNS host-name decoder written as a state machine could not be verified within 900 s and ran 5.6 times slower than handwritten C (EXP-007, repeated in EXP-008); adding a bounded fold to the core made it verifiable in 16.9 s at 1.02 times handwritten C, yet Dafny verified it in 2.95 s with fewer lines (EXP-009); and a TCP options parser passed its registered criteria only on verification time (EXP-010). These results establish feasibility in a restricted domain, not general soundness, ergonomic superiority, or performance dominance. We identify design and measurement weaknesses that must be addressed before stronger comparative claims. **Keywords:** domain-specific languages; binary parsing; formal verification; SMT; empirical software engineering; compilation to C.

1 Introduction

Protocol decoders routinely depend on tight constraints on byte lengths, encodings, integer widths, and reserved values. Testing can reveal failures but does not in itself establish equivalence for all bounded inputs. Formal methods provide alternative guarantees, although practical implementations vary in the annotations and toolchain components required. EFCH Lang investigates a restricted combination: a specification in the EFCH core, an imperative bounded program in a separate source language (`.efl`), translation to C without an LLM at compilation time, and an equivalence-checking workflow.

The research question is deliberately limited: *Can the prototype express and verify small standards-derived binary decoders while generating C with runtime costs competitive with handwritten code, and what advantages remain after comparison to mature verification tools?* We combine a first eight-function evaluation (EXP-005, 21 October 2026) with a distinct follow-up (EXP-015, 24 October 2026). The latter is not a statistically independent random sample of all parser tasks; its value is in reporting a separately recorded protocol and different authored baselines. To avoid selective reporting, Section 5 summarizes every other preregistered experiment of the same line (EXP-004, EXP-007–EXP-010), including those that failed their criteria.

Our contributions are: (i) an explicit, reproducible characterization of a bounded application domain; (ii) original measurements and protocol-level decisions for two evaluations; (iii) a sensitivity analysis showing how baseline design changes comparative claims; and (iv) a candid account of unsolved verification and generalization risks. The core contribution is empirical feasibility evidence, not a new proof calculus.

2 System and Scope

The EFCH core encodes computation and expected behavior; EFCH Lang defines implementation expressions

with bounded operations and translates them to C. Its verifier checks correspondence between the implementation and specification, commonly through the Z3 SMT solver [1]; difficult original cases can require bounded model checking. This is not an assertion that the compiler itself is mechanically verified, nor that verified source programs imply correct behavior under an unverified compiler and runtime without additional assumptions.

The target tasks were pure, small binary functions with bounded input sizes (at most 60 bytes in EXP-005). Features of general-purpose languages, concurrency, dynamic stateful systems, and protocol implementations with unbounded resources were outside the evaluated scope.

3 Study Design

3.1 EXP-005: eight standards-derived functions

The experiment recorded its acceptance conditions before implementation. The corpus comprised QUIC variable-length integer, HPACK prefixed integer, CBOR item header, UTF-8 first code point, base64 block, IPv4 header validation, Bitcoin CompactSize, and MessagePack integer. The experiment excluded certain earlier language-design examples. Each task was represented as an EFCH specification and as an EFCH Lang program, handwritten C, Rust plus Kani harness, Dafny, and Go (the latter for performance comparison rather than formal proof).

The preregistered criteria were: V1, express at least six of eight functions; V2, verify at least six of eight equivalences; V3, generated-C/handwritten-C geometric-mean time ratio at most 1.10 with no task above 1.25; and V4, pass at least one of three specified advantages against *each* of Rust–Kani and Dafny, based on source-line count, number verified, or verification-time ratios. The original verification budgets were 60 s for Z3 and 600 s for CBMC, Kani, and Dafny, with a reported 2.5 GB memory bound.

The measurement process recorded standards-based test vectors and differential checks against the EFCH specification interpreter; the runtime harness used three workload seeds, 10,000 inputs per seed, and 15 interleaved measurement repetitions. The EFCH-generated C and handwritten C used `-O3 -march=native`. Rust used optimized native-target compilation. Go used a separate harness, so performance comparisons to Go are less tightly controlled than the C comparison.

3.2 EXP-015: two further functions

This follow-up specified an IPv4 header with options, checksum, and metadata extraction, and a Protocol Buffers-style unsigned 64-bit varint decoder. The roles were separated across AI agents: one agent (Claude Opus), which knew the backend-optimization hypothesis under test, wrote the specifications and the EFCH programs; a second agent wrote the handwritten C baselines; and a third wrote the test vectors from the RFCs and a Go reference. The workload generator was fixed before measurement and stratified into random, valid, and perturbed inputs. Verification and benchmark acceptance limits were preregistered; in particular, the runtime ratio had to meet the M9 criterion with upper target 1.25. EXP-015 used 110,000 differential inputs per function and included 67 IPv4 and 58 varint specification vectors.

Table 1: EXP-005 originally recorded outcomes. Ratios are not cross-machine or population claims.

Metric	Recorded outcome
Expressions admitted	8/8
Equivalences verified	8/8
Runtime geometric mean, EFCH C / handwritten C	0.90
Largest single runtime ratio	1.035
Relative source lines vs Rust–Kani	0.61 (0.70 under a later formatting sensitivity)
Relative source lines vs initial Dafny	0.90
Verification at least 2× faster on individual tasks	7/8 vs Kani; 7/8 vs initial Dafny
Original IPv4 EFCH proof workflow	70.2 s with CBMC fallback

4 Results

4.1 Initial eight-function study

Table 1 reports only the aggregated values recorded in the EXP-005 report. All eight functions were expressible and returned a proved verification status under the procedure. The initially evaluated generated-C runtime ratios were near handwritten C. The IPv4 case required a slower CBMC route after Z3 was unable to discharge the obligation within the initial strategy.

The recorded original decision was "viable" *under the predeclared rule*. This should not be confused with superiority over existing tools. The verifier counts do not control for equivalence of proof obligations, the authored reference structure, or the total effort required to establish trustworthy specifications.

4.2 Adversarial review and sensitivity

An AI-agent audit found a CBOR malformed-input error, subsequently corrected in the recorded study, and several unresolved measurement and implementation concerns. In particular, the original IPv4 workload contained no valid headers among 30,000 generated inputs, making its fast rejection path unrepresentative of realistic parsing. The relative line-count advantage against Kani was sensitive to formatting and reference style.

Table 2: EXP-015 reported results; runtime ratio is EFCH-generated C / independently authored C.

	IPv4 options	Varint-u64
Test vectors passing	67/67	58/58
Differential inputs	110,000	110,000
Disagreements reported	0	0
Specification check	1.2 s	0.28 s
Program verification	1.4 s	0.42 s
Runtime ratio	1.027	0.966
Reported M9 band	[0.96, 1.12]	[0.90, 1.03]
Program lines / C lines	45/59	12/20
Workload accepted	0.44	0.74

A follow-up Dafny construction returning `Spec(...)` verified all eight functions in 2.1–2.5 s each, invalidating a broad claim that EFCH requires fewer source lines. The reported EFCH/Dafny source-line ratio ranged from 0.87 for specification-only comparisons to 1.56 when EFCH specification plus implementation was compared with the Dafny specification-only variant. These are alternate accounting conventions, not mutually contradictory estimates of one fixed quantity.

Executable Dafny specifications translated to Go were slower than C produced from the EFCH specification on the measured workloads; the study reports a geometric mean factor of about 165, but that comparison includes differences in arithmetic types, immutable sequences, target runtime, and representation. The stronger comparator—Dafny with bounded types and an optimized imperative representation—was tested only for QUIC, not across the corpus. Thus 165 is *not* evidence of inherent superiority over efficiently implemented Dafny. A subsequent EFCH verification-by-cases modification lowered the IPv4 check to a reported 7.4 s with Z3; this was exploratory and not used to retroactively revise the registered EXP-005 decision.

4.3 Follow-up EXP-015

Both tasks satisfied their registered correctness, verification, and performance criteria. However, the tested field decomposition optimization did not produce a measurable IPv4 speedup (ratio 1.007 with reported band [0.95,1.08]); in varint it was not activated. Consequently, these results support prototype feasibility, *not* attribution of speed to that optimization.

5 Other Experiments in the Same Line

EXP-005 and EXP-015 are the favourable end of a longer sequence. Table 3 summarizes the remaining preregistered experiments; all values are those recorded in the corresponding protocols. "Effort" is authored source lines relative to the comparator.

Two observations follow. First, the EXP-005 viability did not extend unchanged to stateful decoders: EXP-007 failed verification, performance and effort, and EXP-008 indicated that, for verification, the failure came not from how the program was written but from the 32-step unrolled specification the core then required. Second, the bounded fold repaired verification and performance for that task, but it was designed while looking at it (a second-attempt bias), and even then Dafny verified the same function faster and with fewer lines. In EXP-010, the only advantage over Dafny is verification time, and it is partly explained by the program's step function transcribing the specification's, so the congruence check proves little more than the translation; an adversarial review confirmed the verification-time advantage (about 1.0 s including the specification check, against 2.6–2.9 s for Dafny) but not robust performance. The fold-based experiments were written by the same agent that designed the fold.

6 Interpretation and Threats to Validity

Benchmark construction. EXP-005 tasks were selected from published standards, but code and baseline implementations were predominantly produced by the same AI system involved in the prototype. This creates risks of correlated mistakes and biased implementation choices. EXP-015 reduced direct cross-arm exposure using separate agents, but agent independence does not imply human-independent scientific replication.

Correctness. "Proved" is the status of a particular formal check under specified assumptions; it does not establish soundness of every toolchain layer. Differential tests covered finite input samples, not all concrete execution contexts. A subtle checksum mutant remained unresolved under one later Z3 strategy rather than being classified as correct.

Performance. Shared hardware, compiler optimizations, small tasks, benchmark mix, and near-zero times limit extrapolation. EXP-005's unstratified IPv4 workload was especially problematic. Confidence bands for EXP-015 overlap unity, so apparent few-percent advantages must not be described as robust speedups.

Comparator fairness. The initial Dafny and Kani baselines did not prove the same proposition with identical intermediate models, arithmetic encodings, or representation choices. A fair claim of end-to-end advantage requires comparably optimized implementations, aligned properties and budgets, and external reproduction. Source lines authored by an agent measure artifact size, *not* human engineering time.

Selective reporting. The two studies in the title were chosen for this manuscript after their results were known. Section 5 reports the remaining experiments of

Table 3: Other preregistered EFCH Lang experiments. Runtime ratio is EFCH-generated C / handwritten C.

Exp.	Task	Verification	Runtime and effort	Registered outcome
EXP-004	varint, framing (2 examples)	both equivalences proved	within ±2% of handwritten C at `-O3`; no performance advantage	predictions met
EXP-007	DNS host name in wire format (state machine over a buffer)	*unknown*: no result within 900 s; Dafny proved in 2.8 s	5.6× slower (workaround removed early exit); effort 2.08× Dafny	**not met**
EXP-008	same task, rewritten program and specification	*unknown* (880 s); the 32-step unrolled specification itself not proved well-defined	not measured (C is only run after proof); effort 1.96× Dafny	**not met**
EXP-009	same task, with a bounded `fold` added to the core	proved in 16.9 s (fold congruence); Dafny proved in 2.95 s	1.02; effort 1.45× Dafny (71 vs 49 lines), 0.79× Rust–Kani	**not met** (only against Dafny)
EXP-010	TCP options (task reserved for the fold, not used to design it)	proved in 0.61 s; Dafny 3.4 s; Kani unknown at 600 s	1.21 (threshold 1.25; one of three seeds at 1.251, with background load); effort 1.59× Dafny	met at the boundary; against Dafny only via verification time

the line so that readers can weigh them; it does not remove the selection effect. **Scope.** Ten evaluated function specifications across two studies do not constitute ten independent families: IPv4 and varint concepts have prior overlap with development examples. Only small bounded decoders were studied, on a single machine. No external user study, production maintenance study, or longitudinal defect escape analysis was conducted.

7 Related Work

Dafny offers an integrated language and automatic verification workflow with specifications, program contracts and mathematical reasoning [2]. Kani applies bounded model checking to Rust programs and user assertions [3]. Z3 supplies SMT reasoning across theories including bit-vectors [1]. CBMC supports bounded model checking of C programs [4]. Closer to the present domain, EverParse and its 3D frontend generate formally proven binary format parsers from a data description language and have been deployed to harden attack surfaces [10]; 3DGen uses AI agents to propose such descriptions, with the verified toolchain restricting what is accepted [11]. Narcissus derives correct-by-construction encoders and decoders from binary format specifications in Coq [12]. Kaitai Struct generates parsers from declarative format descriptions, without, as far as we found, a formal proof of the generator [13]. These are direct antecedents of a restricted language for verified binary decoding; we do not claim the combination studied here is new, and none of them was included as a comparator. These systems are relevant comparators, not technologies the present paper displaces. In particular, a domain-specific source language may reduce domain-specific syntactic complexity while leaving the foundational assurance to well-established solvers and modeling choices.

8 Reproducibility, Ethics, and Data Availability

The primary archival records are the EXP-005 and EXP-015 protocols and result reports in the private research repository `https://github.com/venture-synaptic/efch-research`. As of manuscript preparation, access to that repository and complete reproducibility artifacts is not publicly guaranteed. A public, version-locked artifact package containing sources, baselines, exact toolchain versions, scripts, raw observations, verification logs, and checksums is *planned but not yet released*. No public replication should be claimed until this package has been deposited and independently checked.

AI coding assistants contributed source code, benchmark material, and adversarial review under the researcher's direction. These systems are not listed as authors. The named author is responsible for interpretation and verification of provenance; the present manuscript has not undergone independent human peer review. The results here are extracted from the repository's documented experiments rather than rerun for this manuscript.

9 Conclusion

The tested EFCH Lang prototype expressed and formally checked a narrow set of standards-derived binary decoding functions and generated C code competitive with handwritten implementations for the measured workloads. Subsequent baseline strengthening removed or

narrowed several initially apparent advantages, and a stateful decoder (EXP-007/EXP-008) failed until a fold was added to the core, after which Dafny still verified it faster and with fewer lines (EXP-009). EXP-015 corroborated local feasibility on two further functions but did not validate the hypothesized backend optimization. The defensible claim is *feasibility under bounded conditions*, not a new general-purpose verification advantage. Future work should prioritize balanced valid-input workloads, aligned strong Dafny and Rust baselines, public artifacts, and independent human validation rather than language feature expansion for its own sake.

References

[1] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," *TACAS*, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24.

[2] K. R. M. Leino, "Dafny: An Automatic Program Verifier," *LPAR*, 2010. https://www.microsoft.com/en-us/research/publication/dafny-an-automatic-program-verifier/.

[3] The Kani Rust Verifier, "User and developer documentation." https://model-checking.github.io/kani/. Accessed November 2026.

[4] The CBMC Project, "CBMC: Bounded Model Checking for C and C++." https://www.cprover.org/cbmc/. Accessed November 2026.

[5] J. Iyengar and M. Thomson, "QUIC: A UDP-Based Multiplexed and Secure Transport," RFC 9000, 2021. https://www.rfc-editor.org/rfc/rfc9000.

[6] C. Bormann and P. Hoffman, "Concise Binary Object Representation (CBOR)," RFC 8949, 2020. https://www.rfc-editor.org/rfc/rfc8949.

[7] R. Peon and H. Ruellan, "HPACK: Header Compression for HTTP/2," RFC 7541, 2015. https://www.rfc-editor.org/rfc/rfc7541.

[8] F. Yergeau, "UTF-8, a transformation format of ISO 10646," RFC 3629, 2003. https://www.rfc-editor.org/rfc/rfc3629.

[9] S. Josefsson, "The Base16, Base32, and Base64 Data Encodings," RFC 4648, 2006. https://www.rfc-editor.org/rfc/rfc4648.

[10] *Hardening Attack Surfaces with Formally Proven Binary Format Parsers*, PLDI 2022. https://project-everest.github.io/assets/EverParse3D.pdf.

[11] S. Fakhoury et al., "3DGen: AI-Assisted Generation of Provably Correct Binary Format Parsers," preprint, arXiv:2404.10362, 2024.

[12] B. Delaware et al., "Narcissus: Correct-by-Construction Derivation of Decoders and Encoders from Binary Formats," *ICFP*, 2019.

[13] Kaitai Struct, "User Guide." https://doc.kaitai.io/user_guide. Accessed November 2026.